SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – ARCHITECTURE / MODELING (SECURE INFRASTRUCTURE)

Course Code:	CSA5803	**Course Title:**	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO4 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2
Weightage:	30% of CIA	**Total Marks:**	100
CO4 Statement:	Design secure DevSecOps infrastructure by developing architecture diagrams, modelling deployment workflows, Identity and Access Management (IAM) relationships, and GitOps workflows.		
Student Name:	Syed Saniya Iqbal	**Reg. No.:**	192465043
Section Batch:	CSE-Cyber Security	**Date:**	25/08/2026

Instructions to Students

- Design all architecture diagrams, deployment workflows, IAM relationships, and GitOps workflows originally and clearly.
- Use DiagrammeR or Draw.io for architecture modelling and maintain consistent labels, trust boundaries, data flows, and security controls.
- Clearly represent IAM roles/permissions, Infrastructure as Code (IaC), GitOps flow, and DevSecOps security integration wherever applicable.

- Include editable source files for diagrams, IaC/configuration files, methodology documentation, and all supporting project artifacts.
- Submit the GitHub repository link and final PDF report through Google Classroom before the specified deadline; verify that the repository is accessible and complete.

Question Type	Focus Area	Questions	Marks
ARCHITECTURE / MODELING	Design and Application Based	Q1	Q1 – 100
GRAND TOTAL:		**100 Marks**	

Design Secure DevSecOps Infrastructure

1. Introduction

A secure DevSecOps infrastructure integrates development, security, operations, Identity and Access Management (IAM), Infrastructure as Code (IaC), CI/CD, and GitOps into one workflow. The proposed architecture protects the application from the development stage until deployment and monitoring. Security checks are automatically performed during coding, building, testing, container creation, and deployment.

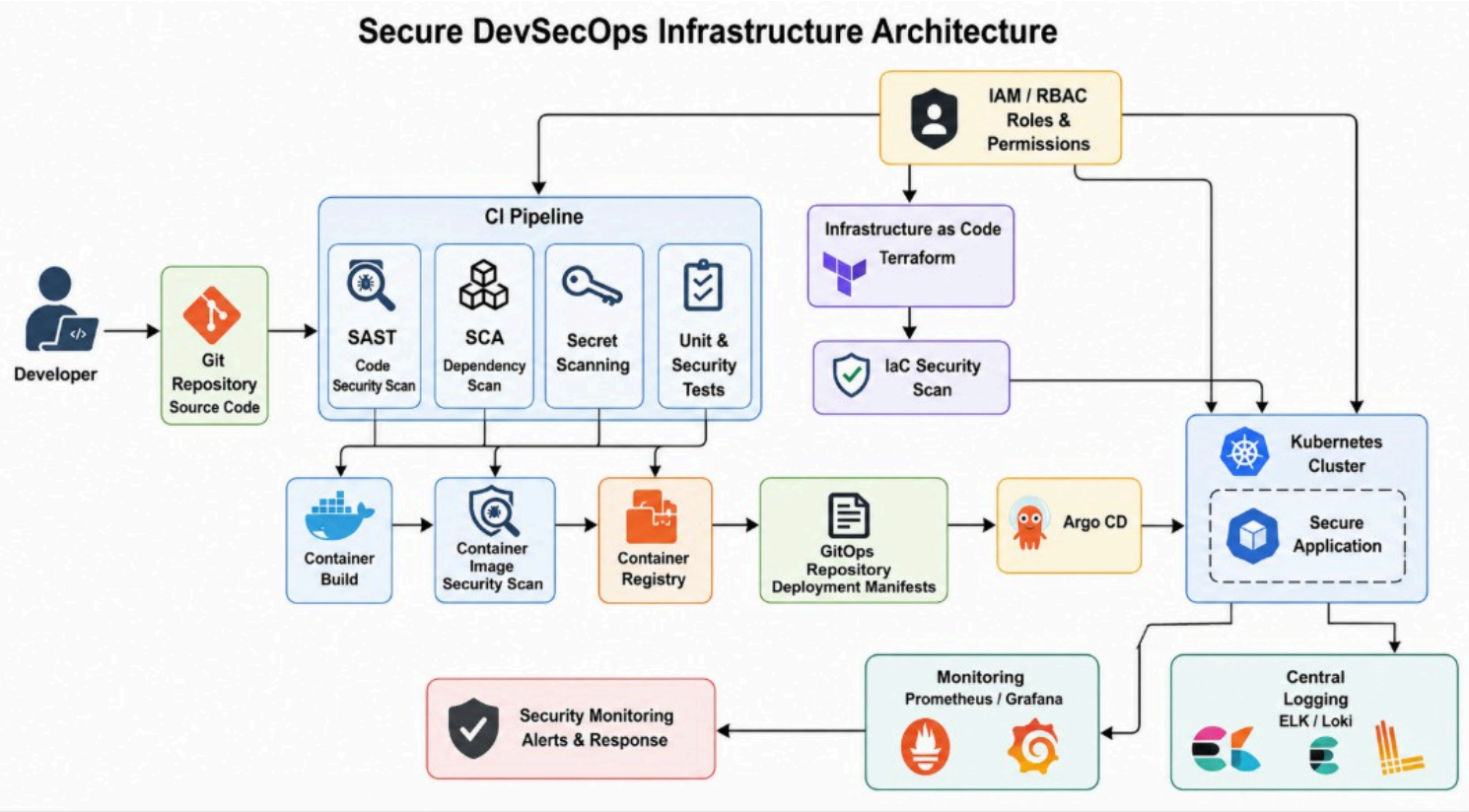
Main objectives

1. Secure source code and Git repositories.
2. Automatically test application security.
3. Scan dependencies and container images.
4. Use IAM with minimum required permissions.
5. Manage infrastructure using IaC.
6. Use GitOps for controlled deployment.
7. Monitor deployed applications continuously.
8. Maintain clear trust boundaries and data flows.

2. Proposed Secure DevSecOps Architecture

The architecture contains the following major components:

- Developer
- Git Repository
- CI/CD Pipeline
- SAST
- SCA
- Secret Scanning
- Unit Testing
- Container Build
- Container Image Scanner
- Container Registry
- Infrastructure as Code
- Terraform
- GitOps Repository
- Argo CD
- Kubernetes Cluster
- IAM
- Monitoring and Logging
- Security Operations



Explanation

The developer first commits code to Git. The CI pipeline automatically performs:

- SAST
- SCA
- Secret scanning
- Unit testing

Only code that passes the required security checks continues to the container-building stage.

The container image is scanned before being stored in the container registry. Terraform manages infrastructure securely through IaC. Argo CD continuously compares the Kubernetes environment with the approved GitOps repository and deploys the required configuration.

3. Trust Boundaries

The architecture should clearly separate trusted and less-trusted areas.

Trust Boundary 1 – Developer Environment

Components:

- Developer workstation
- IDE

- Local source

code Security controls:

- Multi-factor authentication
- Endpoint protection
- Git authentication
- Secret scanning

Trust Boundary 2 – Source Control

Components:

- Git repository
- GitOps

repository Security

controls:

- Branch protection
- Pull-request approval
- Signed commits
- Access control
- Audit logs

Trust Boundary 3 – CI/CD Environment

Components:

- CI runner
- SAST
- SCA
- Container

scanner Security

controls:

- Temporary credentials
- Restricted permissions
- Isolated runners
- Security gates

Trust Boundary 4 – Production

Components:

- Kubernetes

- Application
- Database
- Service

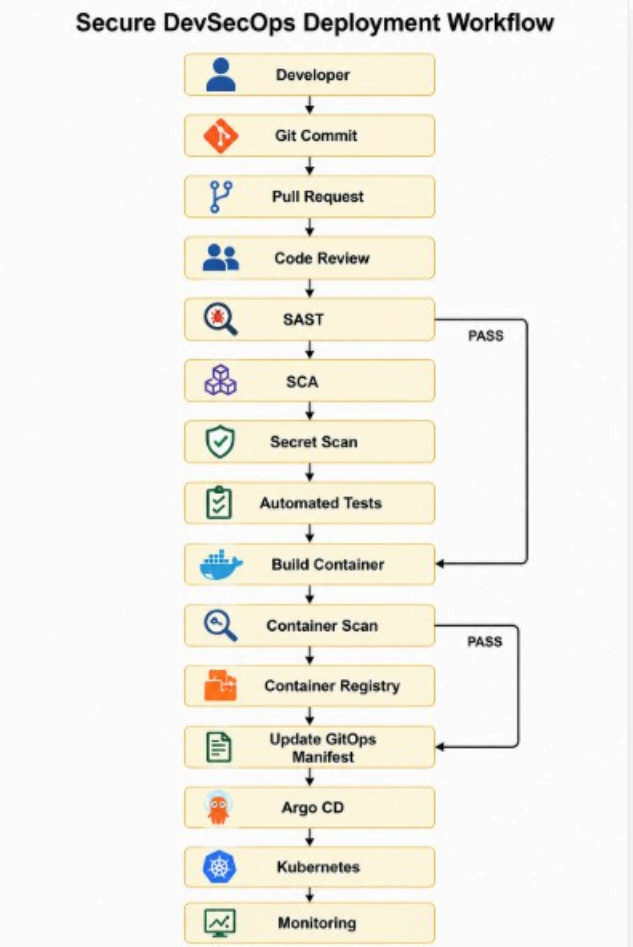
s Security

controls:

- RBAC
- Network policies
- Secrets management
- Container security
- Runtime monitoring

4. Secure Deployment Workflow

The deployment workflow describes how code moves from the developer to production.



Deployment Process

Step 1 – Code Development

The developer writes application code and commits it to the Git repository.

Example 1: A developer creates a secure login API.

Example 2: A developer fixes an SQL injection

vulnerability. **Step 2 – Pull Request**

The developer creates a pull request instead of directly changing the production branch.

Step 3 – Code Review

Another authorized developer reviews the changes.

Step 4 – SAST

Static Application Security Testing checks the source code for vulnerabilities.

Examples:

- SQL injection
- Hard-coded credentials
- Unsafe functions

Step 5 – SCA

Software Composition Analysis checks third-party libraries.

Example:

Application

↓

Python Flask

↓

Third-party package

↓

Known vulnerability

If a critical vulnerable package is detected, the pipeline stops.

Step 6 – Secret Scanning

The pipeline searches for:

- API keys
- Passwords
- Tokens
- Cloud credentials

Step 7 – Testing

Automated unit and security tests are executed.

Step 8 – Container Build

The application is packaged into a container image.

Step 9 – Container Security Scan

The image is checked for vulnerable packages and unsafe configurations.

Step 10 – Container Registry

Only an approved image is pushed to the registry.

Step 11 – GitOps

The deployment manifest is stored in the GitOps repository.

Step 12 – Argo CD Deployment

Argo CD detects the approved configuration and synchronizes Kubernetes.

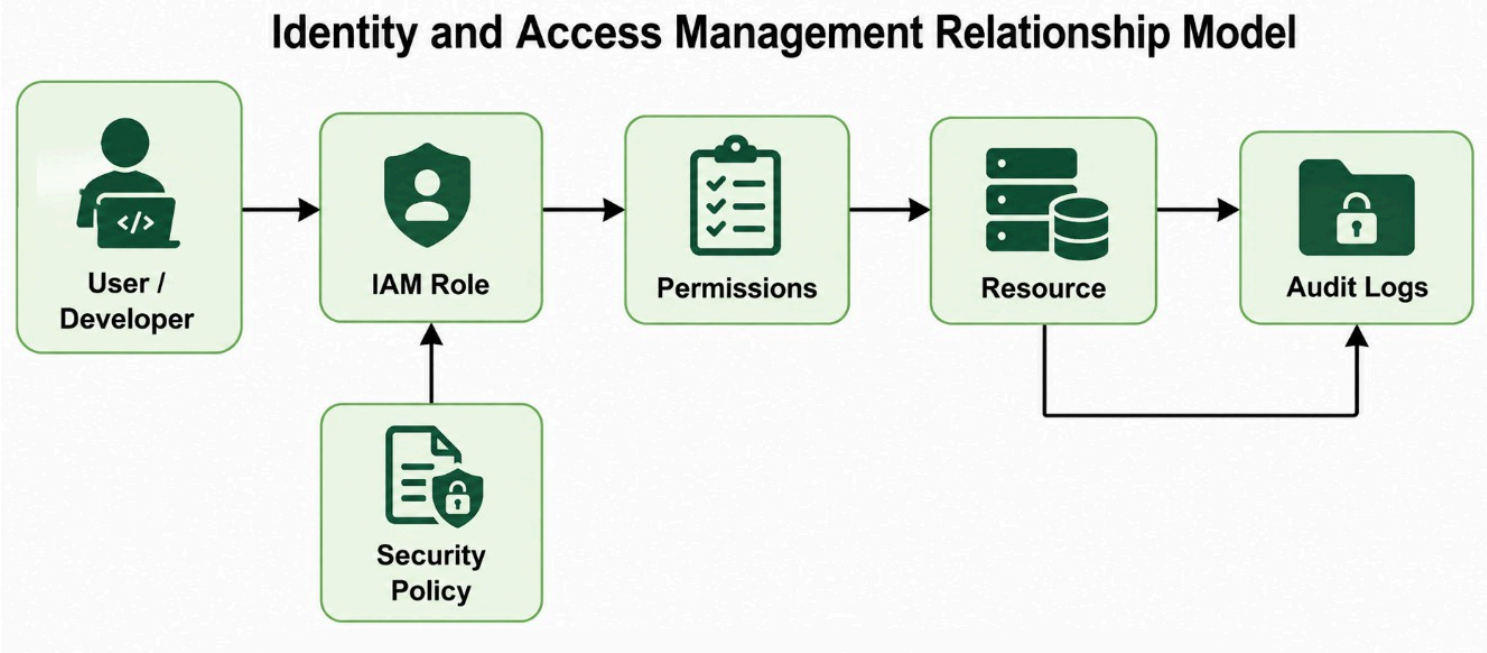
Step 13 – Monitoring

The deployed application is continuously monitored.

5. Identity and Access Management (IAM) Model

IAM controls who can access what and what actions they are allowed to perform.

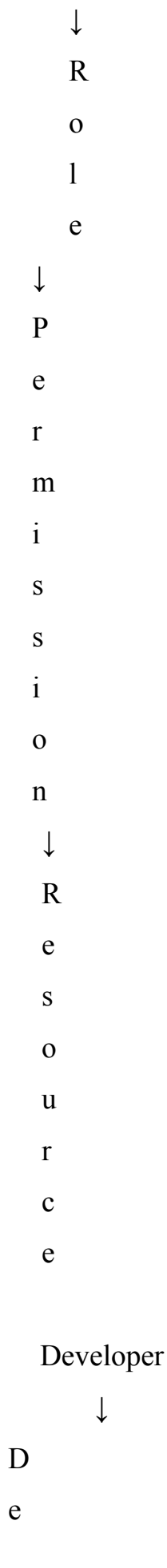
IAM Relationship Diagram



IAM Structure

User

Example 1 – Developer



v rite
e Sou
l rce
o Cod
p e
e ↓
r Dev
R elop
o men
l t
e Rep
↓ osit
Rea ory
d/W

The developer should **not** have production administrator access.

Example 2 – CI/CD Service Account

CI Service Account

↓

CI Role

↓

Build + Scan + Push Image

↓

Container Registry

The CI account should not have unrestricted Kubernetes administrator permissions.

6. IAM Roles

Role	**Main Permissions**
Developer	Read/write development code
Security Engineer	Security tools, reports and policies

Role	Main Permissions
CI Service Account	Build, test and push approved images
Argo CD	Deploy approved manifests
Kubernetes Admin	Cluster administration
Monitoring Account	Read monitoring data
Auditor	Read-only logs and reports

Principle of Least Privilege

Each identity should receive only the permissions required for its task.

Bad design:

CI Pipeline → Administrator → Entire Cloud Account

Better design:

CI Pipeline → CI Role → Container Registry

7. Infrastructure as Code (IaC)

Infrastructure should not be manually configured wherever possible.

Terraform can be used to define:

- Virtual networks
- Kubernetes clusters
- IAM roles
- Security groups
- Databases
- Storage
- Monitoring

infrastructure **Example**

Terraform Structure

infrastructure/

│

├── main.tf

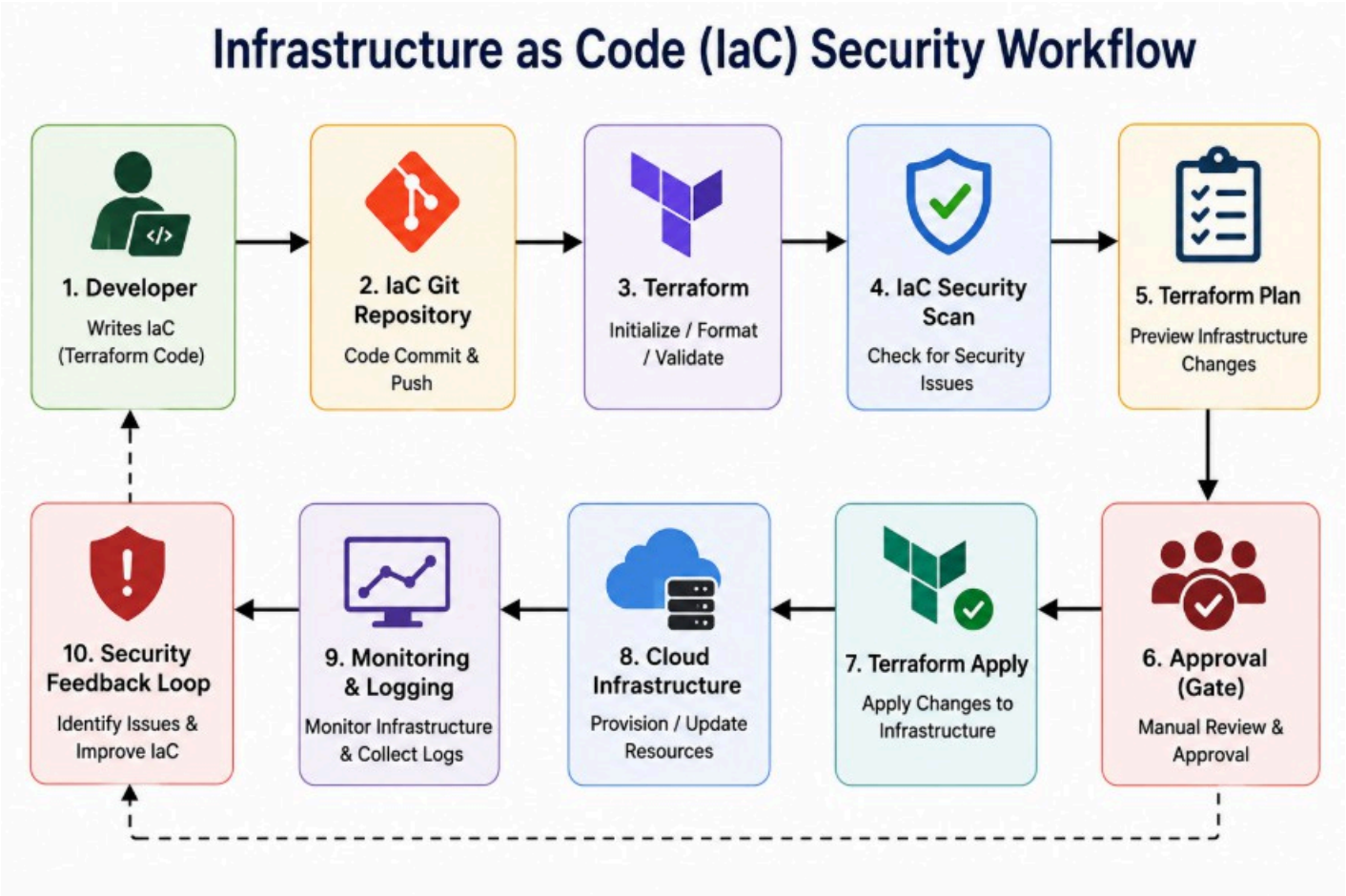
├── variables.tf

├── outputs.tf

├── providers.tf

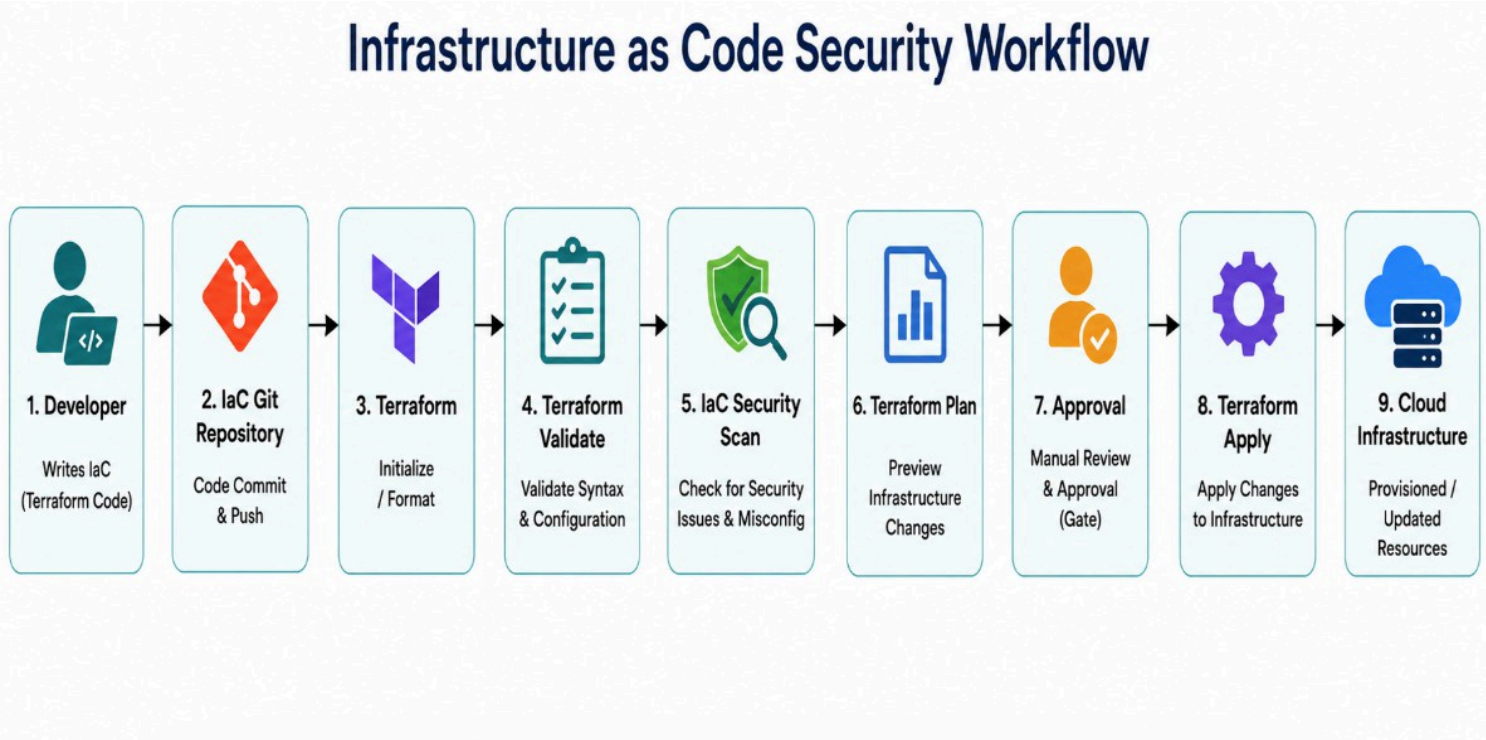
├── iam.tf
├── network.tf
├── kubernetes.tf
└── security.tf

Simple Terraform Example



In a real deployment, additional controls should be applied such as encryption, access policies, logging and versioning.

8. IaC Security Workflow



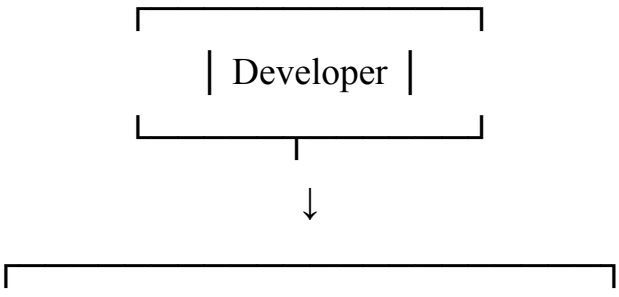
Security checks

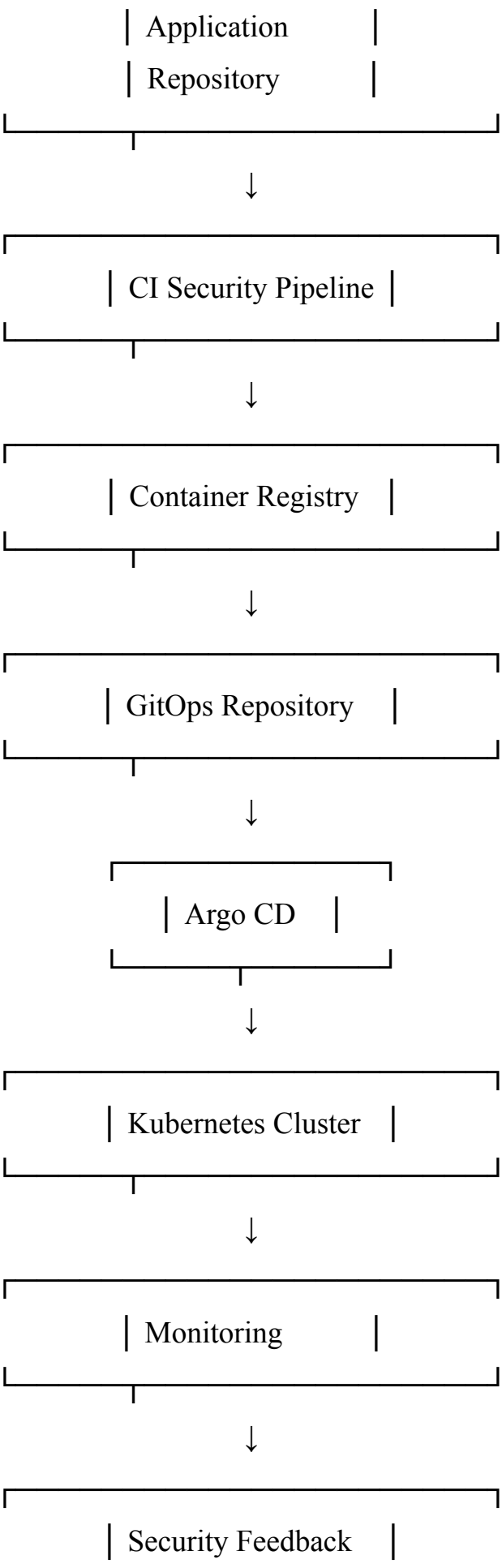
Before infrastructure is deployed:

1. Validate Terraform syntax.
2. Scan IaC configuration.
3. Check IAM permissions.
4. Check network configuration.
5. Check encryption settings.
6. Review Terraform plan.
7. Obtain approval.
8. Apply changes.

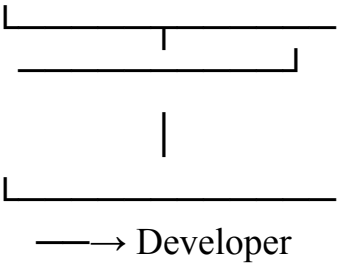
9. GitOps Workflow

GitOps uses Git as the **source of truth for deployment configuration**.




Application Repository
CI Security Pipeline
Container Registry
GitOps Repository
Argo CD
Kubernetes Cluster
Monitoring
Security Feedback

GitOps Process



Developer

↓

A

p

p

l

i

c

a

t

i

o

n

G

i

t

↓

CI Security Checks

↓

C

o

n

t

a

i

Why GitOps is secure

GitOps provides:

- Version history
- Change tracking
- Pull-request review
- Easy rollback

n
e
r
R
e
g
i
s
t
r
y
↓
GitOps Repository
↓
A
r
g
o
C
D
↓
K
u
b
e
r
n
e
t
e
s
↓
M
o
n
i
t
o
r
i
n
g
↓
S
e
c
u
r
i
t
y
F
e
e
d
b
a
c
k

- Controlled deployment
- Auditability

Example 1 – Normal deployment

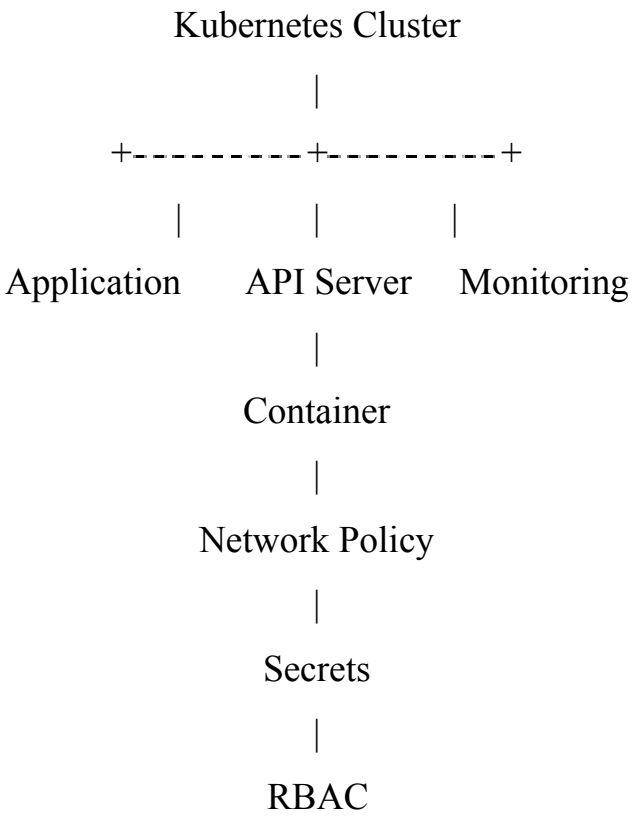
A new approved application version is committed to the GitOps repository. Argo CD deploys it to Kubernetes.

Example 2 – Rollback

If version v2 causes a security or availability problem, the Git repository can be reverted to v1.

10. Kubernetes Security

The production Kubernetes environment should contain multiple security controls.



Important controls

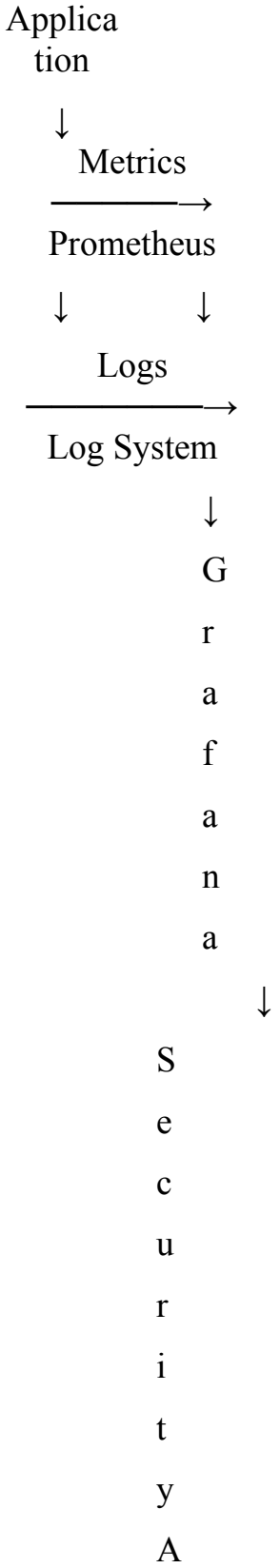
- Kubernetes RBAC
- Network policies
- Namespace isolation
- Container image scanning
- Non-root containers
- Resource limits
- Secrets management
- Pod security controls
- Audit logging

- Runtime monitoring

11. Monitoring and Security Response

Monitoring is required after deployment.

Monitoring architecture



Monitoring should identify:

- Failed login attempts
- Unusual network traffic
- Container failures
- Unauthorized access
- CPU/memory anomalies
- Suspicious API activity
- Configuration changes

l u
e r
r i
t t
s y
↓ T
S e
e a
c m

12. Security Gates

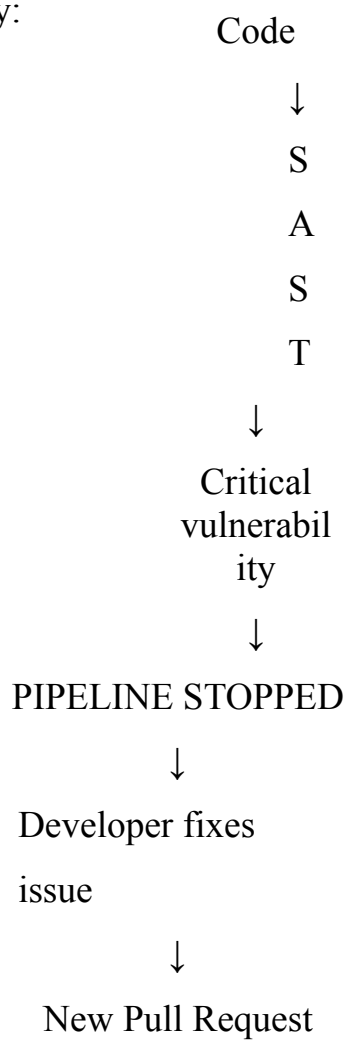
Security gates prevent unsafe code from reaching production.

Stage	Security Gate
Source Code	Secret scanning

Stage	Security Gate
Pull Request	Code review
Build	SAST
Dependency	SCA
Container	Image scanning
Infrastructure	IaC scanning
Deployment	Approval/GitOps
Runtime	Monitoring
Production	Security alerts

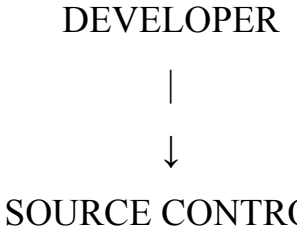
Example

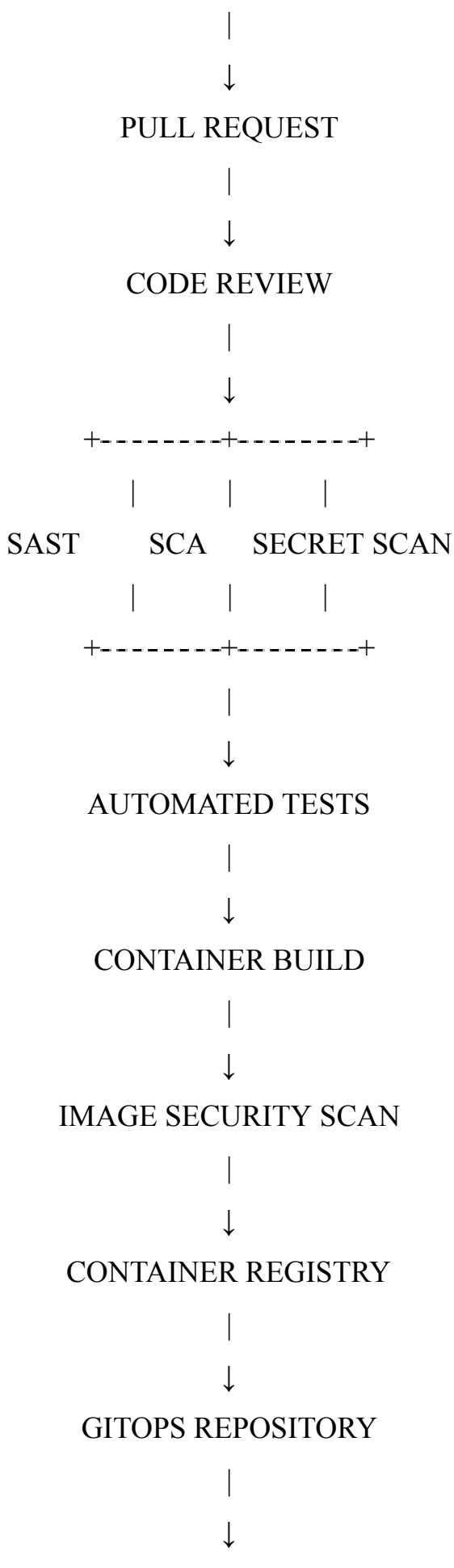
If SAST detects a critical vulnerability:

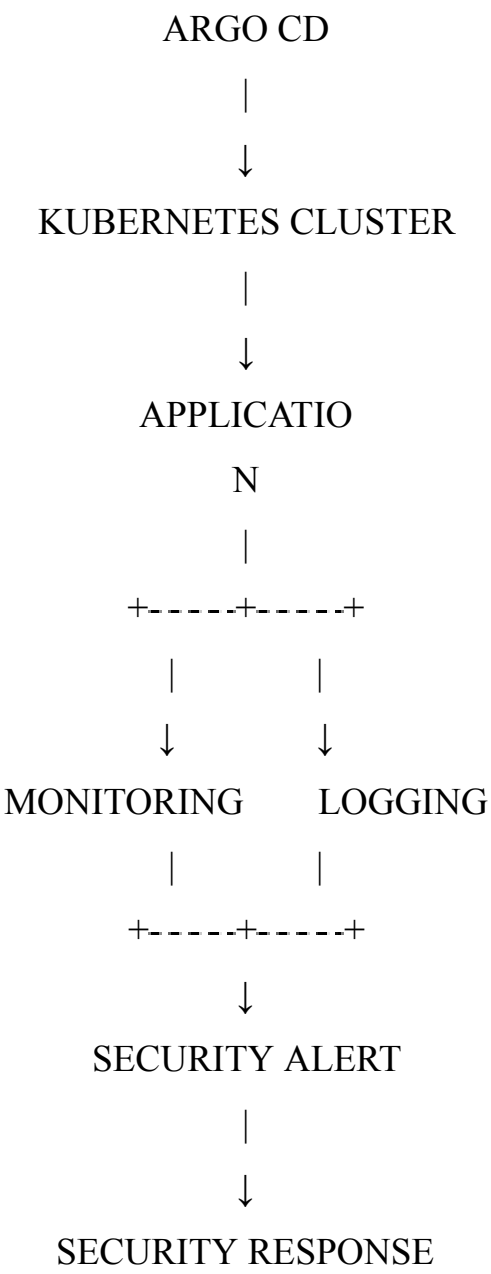


This is better than deploying first and discovering the vulnerability later.

13. Secure DevSecOps End-to-End Workflow


DEVELOPER
SOURCE CONTROL


PULL REQUEST
CODE REVIEW
SAST
SCA
SECRET SCAN
AUTOMATED TESTS
CONTAINER BUILD
IMAGE SECURITY SCAN
CONTAINER REGISTRY
GITOPS REPOSITORY



14. Security Controls Mapping

Security Area	**Control**
Identity	IAM + MFA
Source Code	Branch protection
Code	SAST
Dependencies	SCA
Credentials	Secret scanning
Container	Image scanning

Infrastructure	Terraform + IaC scanning

Security Area	Control
Deployment	GitOps
Kubernetes	RBAC + Network Policies
Secrets	Secrets Manager
Monitoring	Prometheus/Grafana
Logging	Centralized logging
Incident Response	Alerts and response workflow

15. Repository Structure

The GitHub repository should contain editable source files and supporting documentation. secure-devsecops-infrastructure/

```
│
├── README.md
│
├── architecture/
│   ├── architecture.R
│   ├── deployment-workflow.R
│   ├── iam-model.R
│   ├── gitops-workflow.R
│   └── diagrams/
│
├── application/
│   ├── src/
│   ├── tests/
│   └── Dockerfile
│
├── infrastructure/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
```

```
│   ├── iam.tf
│   ├── network.tf
│   └── security.tf
│
├── kubernetes/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── namespace.yaml
│   └── network-policy.yaml
│
├── gitops/
│   ├── application.yaml
│   └── deployment.yaml
│
├── security/
│   ├── sast/
│   ├── sca/
│   ├── iac-scan/
│   └── container-scan/
│
├── monitoring/
│   ├── prometheus/
│   └── grafana/
│
└── documentation/
├── methodology.md
├── security-controls.md
└── deployment-guide.md
```

16. Methodology

The project can be implemented using the following methodology:

Phase 1 – Requirement Analysis

Identify:

- Application requirements
- Infrastructure requirements
- Security requirements
- User roles
- Deployment

requirements **Phase 2 – Architecture Design** Design:

- DevSecOps architecture
- Trust boundaries
- Data flows
- IAM relationships
- Deployment

workflow **Phase 3 – IaC Development** Create

Terraform files for:

- Infrastructure
- IAM
- Network
- Security controls

Phase 4 – Security Integration

Integrate:

- SAST
- SCA
- Secret scanning
- IaC scanning
- Container

scanning **Phase 5 –**

GitOps Create:

- GitOps repository
- Kubernetes manifests
- Argo CD

configuration **Phase 6 –**

Monitoring Configure:

- Metrics
- Logs
- Alerts
- Security

monitoring **Phase 7 –**

Validation Test:

- Unauthorized access
- Vulnerable code
- Vulnerable dependencies
- Invalid IAM permissions
- Unsafe infrastructure
- Failed deployments

17. Security Verification

The architecture should be tested using security scenarios.

Test Case 1 – Hard-coded password

Developer commits password

↓ Secret

Scanner

↓

Secret detected

↓

Pipeline fails

Expected result: Deployment must be blocked.

Test Case 2 – Vulnerable dependency

Developer adds vulnerable library

↓

SCA Scanner

↓

Critical vulnerability detected

↓

Pipeline fails

Expected result: Developer must replace or update the dependency.

Test Case 3 – Unauthorized deployment

Unauthorized User

↓

IAM

Check

↓

Permission Denied

↓

Deployment blocked

Expected result: Unauthorized users cannot deploy to production.

18. Benefits of the Proposed Architecture

1. Security is integrated throughout development.
2. Automated security testing reduces human mistakes.
3. IAM limits unauthorized access.
4. IaC provides repeatable infrastructure.
5. GitOps provides controlled deployments.
6. Kubernetes provides scalable application deployment.
7. Monitoring detects suspicious activity.
8. Git history provides traceability.
9. Rollback becomes easier.
10. Security problems can be detected earlier.

19. Conclusion

The proposed Secure DevSecOps infrastructure combines DevSecOps, IAM, IaC, CI/CD, GitOps, Kubernetes, and continuous monitoring into one secure architecture.

The main security principle is "security at every stage" rather than checking security only after deployment.

The complete workflow is:

Code

↓

Review

↓

SAST + SCA + Secret Scan

↓

Testing

↓

Container

Scan

↓

Registry

↓

GitOps

↓

Argo CD

↓

Kubernetes

↓

Monitoring

↓

Security

Response

This architecture satisfies the required areas of architecture modelling, deployment workflow, IAM

relationship modelling, IaC, GitOps workflow, security integration, trust boundaries, data flows, and continuous monitoring.

Final submission checklist

- Architecture diagram created using DiagrammeR/Draw.io
- Deployment workflow diagram included
- IAM relationship diagram included
- GitOps workflow diagram included
- Trust boundaries identified
- Security controls identified
- IaC files included
- Kubernetes configuration included
- GitHub repository created
- Editable diagram source files included
- Methodology documentation included
- Screenshots/evidence added
- Final PDF report prepared
- GitHub repository link verified before submission

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
IAM Model and Access Control Design	20%	Designs a comprehensive IAM model with clearly defined identities, roles, permissions, authentication and least-privilege access controls	Designs an appropriate IAM model with most roles and access controls correctly represented and only minor gaps	Provides a basic IAM model with limited role definition, permissions or access-control details	Provides an incomplete or inaccurate IAM model with weak or inappropriate access controls

Infrastructure as Code (IaC) Modelling	30%	Accurately models infrastructure provisioning through IaC with reusable, consistent and secure configuration components	Models IaC workflow correctly with appropriate configuration structure and only minor omissions	Provides a basic IaC model with limited automation, structure or security considerations	Provides an incomplete or incorrect IaC model with poor configuration and automation representation
GitOps Workflow and Version Control	20%	Clearly models a complete GitOps workflow including repository changes, review, automated synchronization, deployment and desired-state management	Models the GitOps workflow correctly with most stages represented and minor workflow gaps	Provides a basic GitOps workflow with missing stages or unclear repository-to-deployment relationships	Provides an incomplete or inaccurate GitOps workflow with weak version-control integration

Security Integration and Workflow Automation	20%	Effectively integrates IAM, IaC and GitOps with security validation, approvals, automation and auditable workflow controls	Integrates appropriate security and automation controls with only minor gaps	Shows basic security integration and automation but several controls or workflow relationships are missing	Shows limited or incorrect integration of security controls and workflow automation
Documentation and Workflow Clarity	10%	Clearly documents IAM, IaC and GitOps components and presents the complete workflow with strong technical justification	Provides clear documentation and workflow explanation with minor clarity or justification gaps	Provides basic documentation with limited explanation of workflow components and relationships	Provides incomplete or unclear documentation with little technical justification

Rubrics:

Criteria	**Weightage**	**Q5 (10)**	**Total Marks (50)**
IAM Model and Access Control Design	25%		
Infrastructure as Code (IaC) Modelling	30%		
GitOps Workflow and Version Control	20%		
Security Integration and Workflow Automation	15%		
Documentation and Workflow Clarity	10%		
Total per Question	**100 Marks**		

100